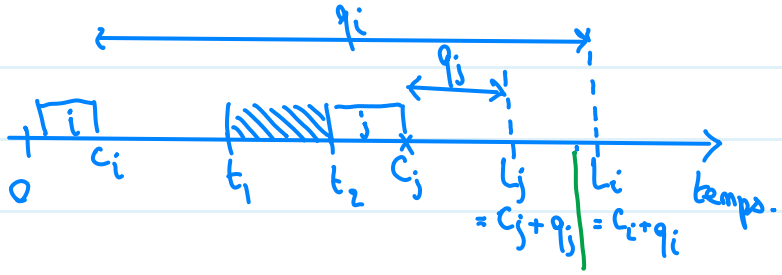
Entrée: 1 machine, n tâches, p_j : durée opératoire de la tâche j
q_j : durée de livraison de la tâche j
$[t_1, t_2]$: indisponibilité, $t_1 < t_2$.

Objectif:

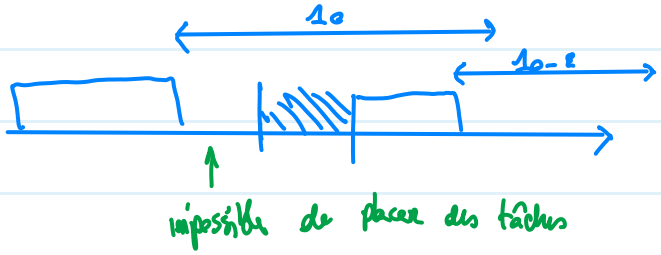


Il faut minimiser le L_{max}, $L_{max} = \max_j \{L_j\}$.

Supposons que $[t_1, t_2]$ n'existent pas, on pourrait ordonnancer le placement selon la règle de Jackson: ordre décroissant des q_j.

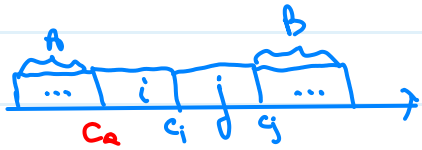
Sans $[t_1, t_2]$, l'algo est optimal. Cependant avec $[t_1, t_2]$, la complexité augmente, ET l'algo n'est plus optimal.

Ex:



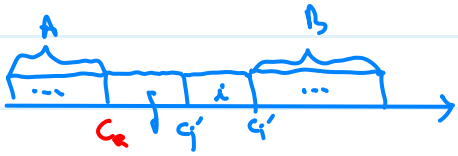
En enlevant $[t_1, t_2]$, <u>prouver que Jackson est optimal:</u>

Par l'absurde, supposons que



et que $q_i < q_j$
et $c_i < c_j$

En inversant i et j, on a:



avec toujours $q_i < q_j$
et $c_i' > c_j'$.

Les tâches $\in A$ et $\in B$ restent inchangées. On note donc $c_j = c_j'$

et $L_{max} = \max\{L_k\}$.

On suppose c_a la date de fin de la dernière tâche de A.

D'une part : i avant j | **D'autre part, j avant i**

On a : $L_i = c_i + q_i = c_a + p_i + q_i$

$L_j = c_j + q_j$.
$= c_i + p_j + q_j$
$= c_a + p_i + p_j + q_j$

On a : $L_{j'} = c_{j'} + q_j = c_a + p_j + q_j$

$L_{i'} = c_{i'} + q_i$
$= c_{j'} + p_i + q_i$
$= c_a + p_j + p_i + q_i$

$L_{j'} \leqslant L_{max}$ car $L_j \leqslant L_{max}$ par supposé, et

$$L_j - L_{j'} \geqslant 0$$
$$\Rightarrow c_a + p_i + p_j + q_j - c_a - p_j - q_j \geqslant 0$$
$$\Rightarrow p_i \geqslant 0$$ ce qui est vrai, ainsi
$$L_j \geqslant L_{j'}$$

Donc : $L_{j'} \leqslant L_j \leqslant L_{max}$.

$L_{i'} \leqslant L_{max}$ car $L_j \leqslant L_{max}$, de la même manière

$$L_j - L_{i'} \geqslant 0$$
$$\Rightarrow c_a + p_i + p_j + q_j - c_a - p_i - p_j - q_j \geqslant 0$$
$$\Rightarrow 0 \geqslant 0$$ vrai donc
$$L_j \geqslant L_{i'} \text{ et } L_{i'} \leqslant L_j \leqslant L_{max}.$$

Ainsi, (L_i', L_j') est optimal, et L_j' étant meilleur, et gagnant encore mieux. Par contradiction, l'algo de Jackson est optimal.

En reprenant la période d'indispo:

Le tout est de d'abord décider si une tâche va à gauche, ou si elle va à droite. Ensuite, chq côté respecte l'ordre de Jackson.

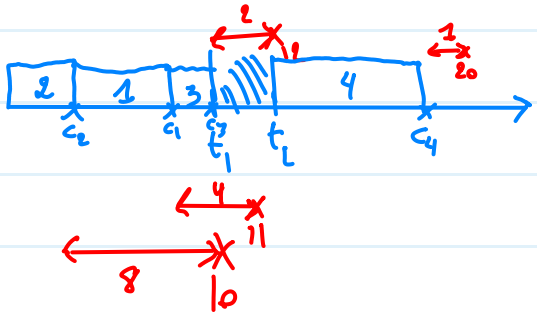
Avec un exemple:

j	1	2	3	4
p_j	5	2	3	7
q_j	4	8	2	1

$t_1 = 10$
$t_2 = 12$

On suppose $\forall i,\ p_i < t_1$
$\sum_i p_i > t_1$

⇒

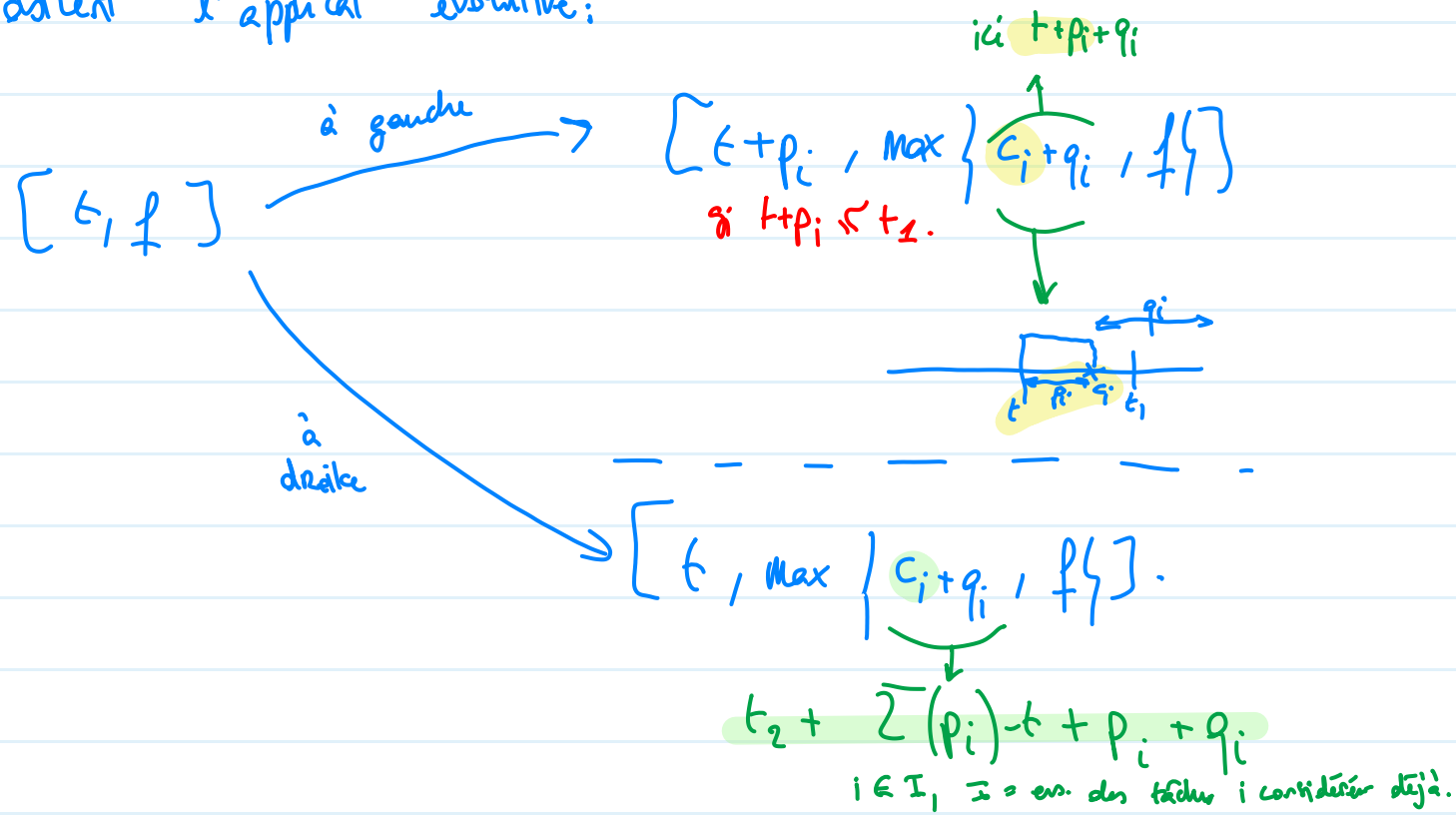


On prend 2 paramètres:

* t: durée totale utilisée avant t_1.
* f: date de livraison de la dernière tâche livrée de l'ordonnancement partiel.

Pour imposer l'ordre de Jackson, à gauche comme à droite, l'entrée doit être triée dès l'ordre de Jackson. Ainsi la décis° ne revient qu'au choix de placer à gauche ou à droite de t_1.

On obtient l'applicat° évolutive :



Maintenant on détermine la complexité de chq étape, en nb d'itérat°s.

E_1 : Tri en ordre de Jackson

E_2 : $E_0 = \{[0,0]\}$.

E_3 : $\forall j \in [\![1, n]\!]$,

initialiser $E_j = \{\}$.

Par chaque état dans E_{j-1}

on crée $[t+p_j,\ \max\{f,\ t+p_j+q_j\}]$ si $t+p_j \leq t_1$

ou

on crée $[t_1,\ \max\{f,\ t_2+(\Sigma p_i)-t+p_j+q_j\}]$

et on les ajoute à E_j.

Supprime E_{j-1}

Par chq valeur $\neq t_1$

garder un état avec $[t, f]$ de E_j avec f minimal.

si cette étape n'est pas là on a $\sum_{i=0}^{n} t^i = t^{n+1} - 1 - O(t^n)$

Si implémentée, ona en ⊕ t_1 facteurs et c'est $O(n \cdot t_1)$

E4 : Une fois E_n produit, les états complets sont générés.
On prend les labels tq on a $\min\{f\}$ de tous les états de E_n. $= \arg\min\{f\}$
$[t, f] \in E_n$.

Pour relaxer les contraintes d'optimalité, on peut être lax. sur certains contraintes pour réduire drastiquement le temps d'exécut°. Le rapport d'approximat° d'un pb de minimisat° tq:

$$r = \max_{\forall I} \left\{ \frac{Algo(I)}{opt(I)} \right\} \quad \text{où} \quad \begin{cases} Algo(I) \text{ est l'approximat°} \\ \text{et } opt(I) \text{ l'optimale} \end{cases}$$

pour une instance I.

En minimisat°, $Algo(I) \geq opt(I)$ donc $\frac{A(I)}{O(I)} \geqslant 1$.
Et, on calcule cela pour $\bar{I}$ = pire instance I, par le pire des cas.

Problème, calculer $opt(I)$ étant trop coûteux, le calcul du rapport reste impossible. On va donc encadrer $opt(I)$:

$$LB(I) \leqslant opt(I)$$

où $LB(I)$ est une borne inférieure estimée. Ainsi,

$$\frac{Algo(\bar{I})}{LB(\bar{I})} \geq \frac{Algo(\bar{I})}{OPT(\bar{I})} = \text{rapport.}$$

$\Rightarrow$ On peut donc estimer le rapport.

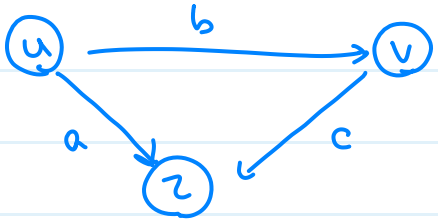
Ex: Problème du voyageur du commerce (Traveling Salesman Problem, TSP).

Entrée : $G = (V, E)$, V = vertices, E = edges. G est complet.
$d(i,j)$: distance de l'arête $(v_i, v_j) = d(j,i)$ (symétrie).
$E \to \mathbb{N}$ (les dist. sont dans $\mathbb{N}$).

Objectif: Trouver un cycle hamiltonien de G tq la somme des distances des arêtes qui participent à ce cycle est minimale.

Notion d'inégalité triangulaire: Une hypothèse faite est que:

$$d(u,z) \leq d(u,v) + d(v,z)$$

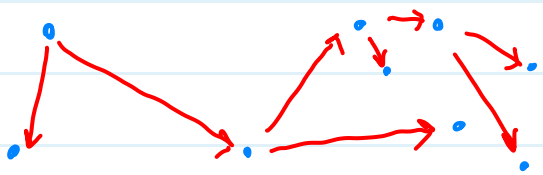


$a \leq b + c$. Nommé ainsi car couvert par les GPS etc.

Reproduisons l'algo. de Christophides.

1. Estimer la borne inférieure LB:
Un cycle hamiltonien, pour $|E| = n$ sommets, on a n arêtes à lier.

On peut trouver une borne par un arbre couvrant de poids minimal:



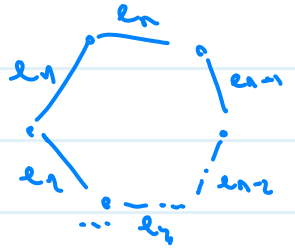
Avec les arêtes de poids minimal. Il y a n-1 arêtes et aucun cycle hamiltonien.

L'arbre = T

W(T) = le poids de T.

⇒ <u>On veut prouver que OPT ≥ W(T) :</u>

Solut° minimale d'un cycle hamiltonien :



n vertices par n edges. $OPT = \sum_{i=1}^{n} d(e_i)$

Supposons e_k, $1 \leq k \leq n$, avec le plus petit poids.

On enlève e_k de la somme.

On obtient $\underbrace{\sum_{i=1}^{n} d(e_i)}_{\text{cycle}} \geq \underbrace{\sum_{i \in [\![1,n]\!] \setminus \{k\}} d(e_i)}_{\text{Chemin hamiltonien}}$

↓

c'est également un arbre couvrant T' qui est différent

⇒ $W(T') = \sum_{i \in [\![1,n]\!] \setminus \{k\}} d(e_i)$ et $W(T') \geq W(T)$ car T' n'est pas forcément de poids minimal.

Donc $OPT \geq W(T') \geq W(T)$. $W(T)$ est bien une LB.

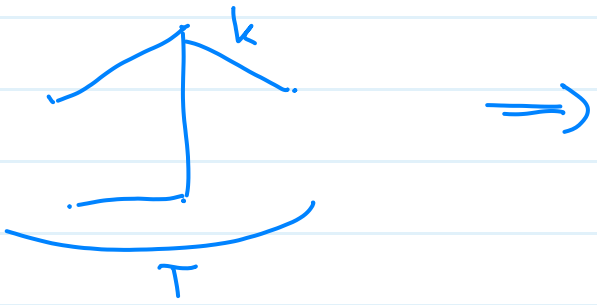
Pour passer d'un ACPM à un cycle hamiltonien, on vient utiliser un cycle eulérien : cycle qui passe 1 seule fois <u>par arête</u>. Pour être un CE, il faut que tous les sommets aient un <u>degré pair</u>.

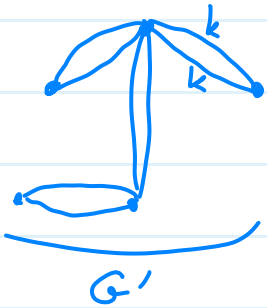
On veut garantir que tous les sommets ont un degré pair.

Le nombre de sommets de degré impair est pair. On peut compléter

En rajoutant une arête par couleur à sommets impair.
↳ On va éviter de faire ça par des pbs de sélect° des points à lier, ni des poids à mettre.

2: On va dédoubler les arêtes:





On dédouble les poids, et chq sommet a un deg. pair.

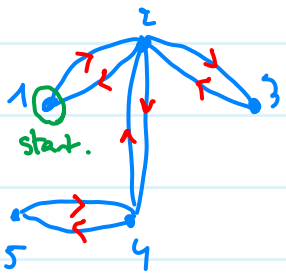
On crée donc les arêtes a_1 et a_2 tq $a_1=(u,v)$ et $a_2=(u,v)$, et $d(a_1)=d(a_2)=d(e)$.

Donc $W(G') = 2W(T)$

Il existe donc un cycle eulérien de G'.
et $2opt \geq W(G')$.

3: Il faut créer un cycle eulérien de G'

Exemple:



CE: 1-2-3-2-4-5-4-2-1.

Ce cycle C a $W(C) = W(G')$.
et $2opt \geq W(C)$

On applique l'inégalité triangulaire, et on tire les k-ièmes occurrences d'un sommet si $k > 1$. Le dernier sommet permet de fermer un cycle

$1-2-3-2-4-5-4-2-1 \Rightarrow 1-2-3-4-5-1 = S$

✓ ✓ ✓ ✓ ✓

hamiltonien

car existe car graphe complet.

et, par inégalité triangulaire, $d(u,v) \leq d(u,k) + d(k,v)$

donc: si S est le graphe engendré, on a $W(S) \leq W(C)$

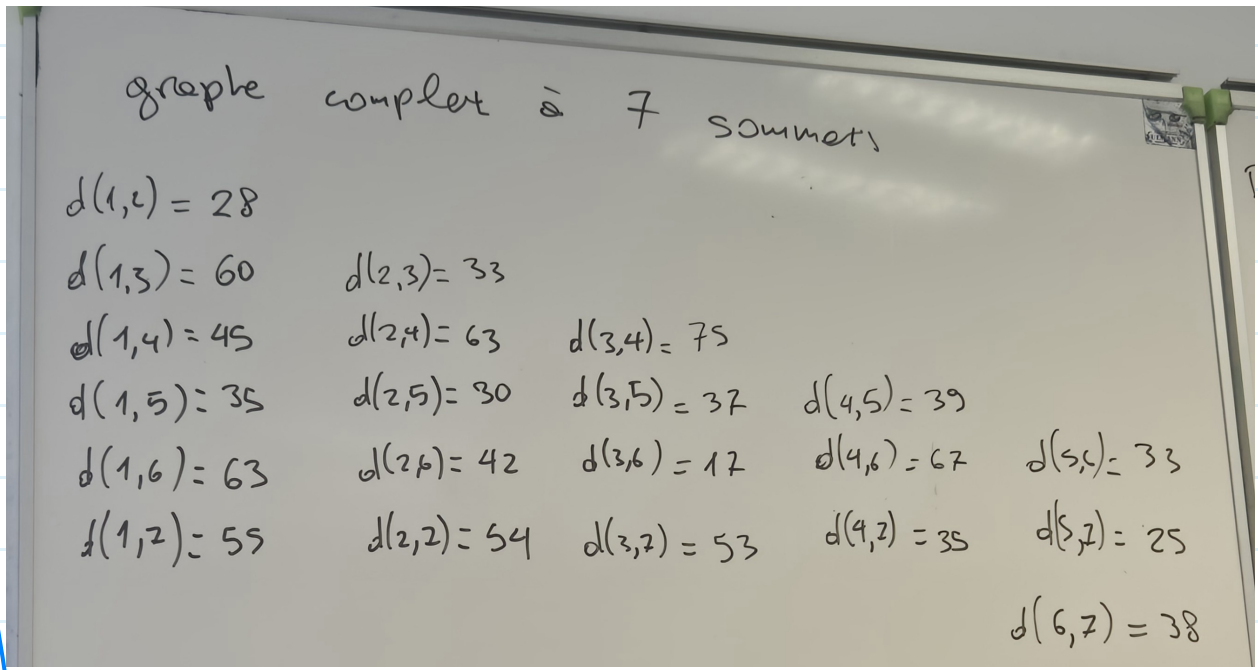
et $W(S) \leq W(C) = W(G') = 2\,W(T) \leq 2\,\text{opt}$

ce qui donne $\frac{W(S)}{\text{opt}} \leq 2$. Le rapport d'approximat° est de 2.

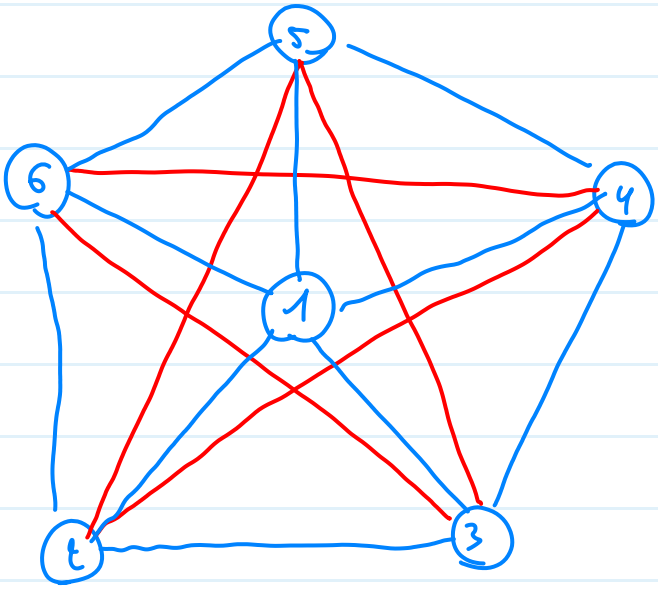
Applicat° de l'algo:

Avec:

pas fait en cours



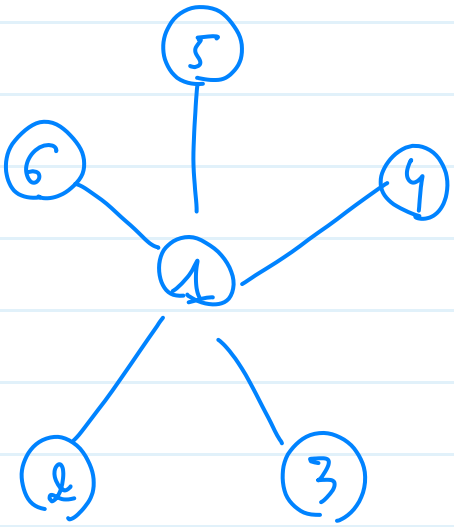
Supposons $G = (V, E)$, G complet.



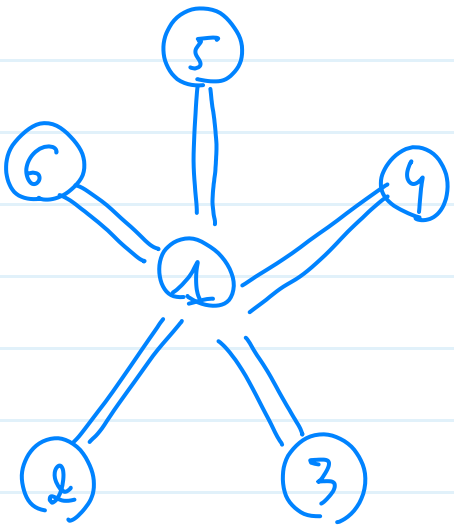
— : 1

— : 2

ACPM :

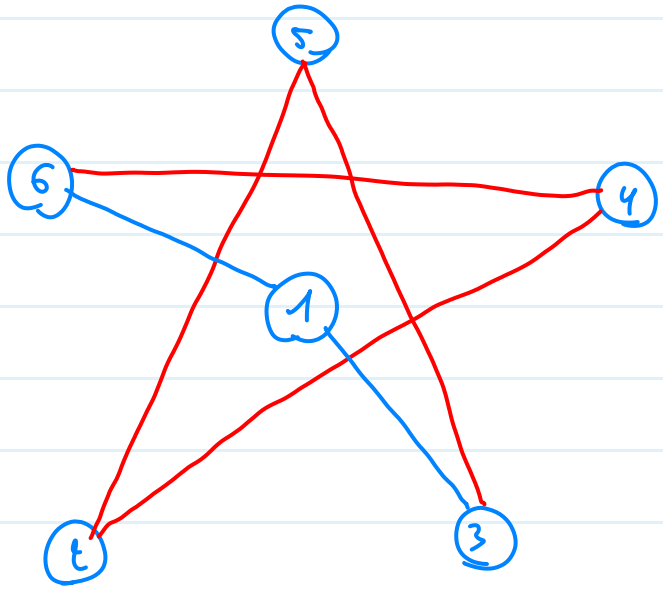


On déduit :



On trouve un cycle: $1-3-\underset{\times}{1}-5-\underset{\times}{1}-2-\underset{\times}{1}-4-\underset{\times}{1}-6-1$

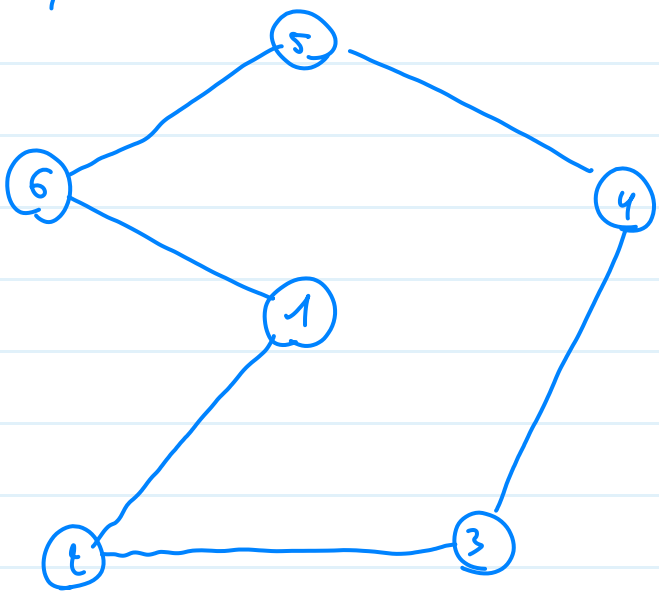
On élimine les doublons: $1-3-5-2-4-6-1$



$\Rightarrow$ 2 arêtes de coût 1 et le reste en 2.

soit $(n-2)\cdot 2 + 2\cdot 1 = 2n-2$.

Or, en optimal, on a:



soit coût n.

On a donc $\frac{A(I)}{OPT(I)} = \frac{2n-2}{n} = 2-\frac{2}{n}$.

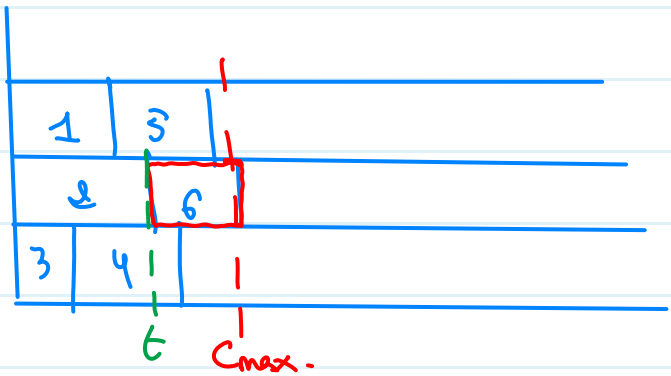
Problème $M||C_{max}$

Pour M machines, on a une borne inférieure à :

utile pour $n \geq m$ (* $OPT \geqslant \frac{\sum p_i}{M} \geqslant \frac{p}{M}$. ↳ les machines terminent en même tps.

utile pour $n < m$ (* $OPT \geqslant p_{max}$ ↳ Pas de préemption, tâches indivisibles.

Exemple: List Scheduling : on place la prochaine tâche sur la 1re machine dispo, aucun tri à faire.



La tâche j est la tâche critique notée j (ici $j=6$).

t : début de la tâche j.

$C_{max} = t + p_j$.

On sait que $t \leqslant \frac{\sum(p_i) - p_j}{M}$

et $C_{max} = t + p_j \leqslant \frac{\sum(p_i) - p_j}{M} + p_j = \frac{\sum(p_i)}{M} + p_j\left(1 - \frac{1}{M}\right)$

On a $\frac{\sum p_i}{M} \leqslant OPT$ et $p_j \leqslant p_{max} \leqslant OPT$

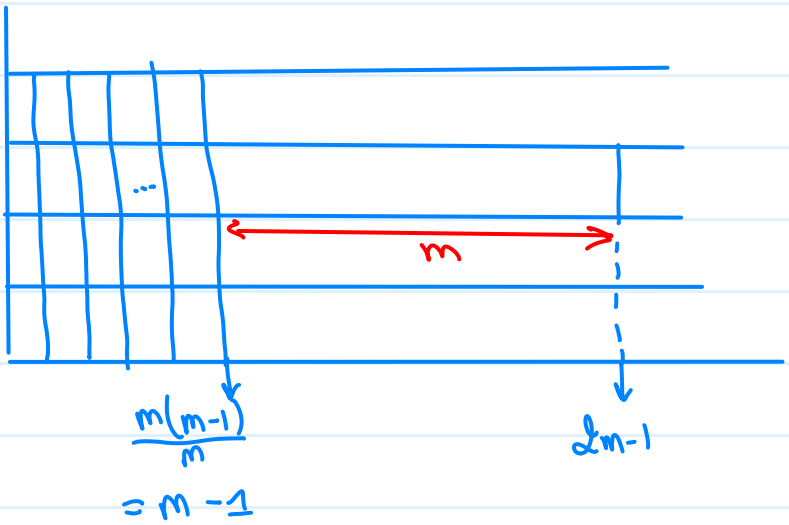
donc $C_{max} \leq OPT + OPT\left(1 - \frac{1}{m}\right) = OPT\left(2 - \frac{1}{m}\right)$.

et $\frac{C_{max}}{OPT} \leq 2 - \frac{1}{m}$.

Pire cas : D'abord les petites tâches et enfin les grandes.

$m(m-1)$ tâches ai $p_j = 1$
et 1 tâche ai $p_j = m$.

$\Rightarrow$



ainsi, $OPT \geq p_{max} = m$
et $OPT \geq \frac{\sum p_i}{m} = \frac{m(m-1)+m}{m} = m-1+1 = m$.